Boogle Dox — AI Workflow & My Engineering Judgment

> Earlier versions: `docs/archive/chatgpt-planning-pack/AI_WORKFLOW.md` (planning draft) and `docs/archive/2026-11-04-repo-docs-before-final/AI_WORKFLOW.md` (build log).

How I approached AI use

I used AI as a development tool, not as a substitute for deciding what the product should be. My first step was deciding what I wanted Boogle Dox to feel like: a recognizable document editor with naming, useful formatting, a familiar menu bar, and reliable sharing. I also decided that reliable saving and access permissions matter more than trying to recreate Google Docs in a few hours.

I wrote my engineering standard into `CLAUDE.md` before the build: inspect before changing, make the smallest safe change, never delete or weaken a failing test to get a green suite, enforce least privilege, and report every result as verified, failed, partial, or not tested. Every AI tool on this project worked under that standard.

Who did what

My decisions (product owner): the product idea, the familiar editor experience, the requirements and scope cuts, owner/editor/viewer sharing, putting permissions ahead of live collaboration, the Collaborative Drafts direction with GitHub-style reviews and team voting, the Boogle Dox name, and the brand (I chose the tagline "Collaboration made easier" over an AI-written one). I approved each milestone, chose when to commit and push, and asked for the git history to be rewritten when a tool added itself as a co-author.

Hands-on steps I did myself: signing in to Supabase and Vercel, creating the reviewer accounts and their passwords, entering the production environment variables in Vercel, and signing in to the live app for testing. The AI tools were not permitted to handle passwords or create accounts.

Tool	What it did
ChatGPT	Helped me organize requirements, evaluate tradeoffs, and draft the planning documents (product requirements, architecture plan, walkthrough outline). I reviewed and customized the product direction.
Cursor (coding assistant)	Built the first local app: Next.js scaffold, Tiptap editor and menus, autosave with revision checks, PGlite persistence, `.txt` / `.md` import, sharing, and the first Vitest suite. It also installed Node.js and Git.
Claude (Claude Code in the Claude desktop app)	Final verification and release: reran lint, type checks, tests, and builds; found and fixed the sign-in bug in the proxy; diagnosed the OneDrive build failure; ran the browser checks; created and configured the Supabase project and deployed to Vercel through my browser, with me doing the sign-ins; recorded the demo GIFs; and rewrote these documents from my planning pack.

The first draft of the hosted Supabase integration was written in an earlier assistant session before the final verification pass. The repository does not record which tool wrote it, so I'm not attributing it to one.

Where AI materially sped up the work

- **Scaffolding and setup:** project setup, Tiptap wiring, and Tailwind styling were generated in minutes instead of hours.
- **Security plumbing:** Supabase row-level-security SQL and the `SECURITY DEFINER` functions for save and sharing, plus a test that runs that SQL against a real Postgres (PGlite) as the `authenticated` role.
- **Debugging:** tracing a build failure (`EPERM` on `.next`) to OneDrive turning build output into cloud placeholders, which Node reported as symlinks.
- **Release work:** clicking through Supabase project setup and the Vercel import while I handled the parts that needed my credentials.

What was changed or rejected

- **A duplicate Underline extension** was removed after the dev server warned about it; Tiptap's StarterKit already includes underline.
- **A proxy that would have blocked every sign-in.** In review, the auth proxy turned out to reject signed-out requests to `/api/auth/sign-in`, so nobody could ever log in. The fix was test-first: a regression test reproduced the 401, then a two-line change made it pass. The test stays in the suite.
- **Unsafe defaults.** When creating the Supabase project, "Automatically expose new tables" was turned off (Supabase's own recommendation) and automatic RLS was turned on, so only explicit grants control API access.
- **An AI co-author line in git history.** A tool appended itself as a co-author on my first commit. I had the commit rewritten so the history shows only me, and later commits were checked on GitHub for any appended trailer.
- **A dashboard tagline** written by AI was replaced with mine.
- **Collaboration scope:** live multi-cursor editing and the edit-lease idea were not built. I kept access control and persistence first, and my Collaborative Drafts idea waits for the next phase.

How correctness was verified

Nothing is marked done because it compiled or because an AI said it worked.

- **Automated:** `npx vitest run` passes 19 tests in 4 files: the real repository and import parser on a temporary database, runtime-mode rules, the hosted RLS SQL, and the auth proxy. `npx tsc --noEmit`, `npx eslint src vitest.config.ts --max-warnings 0`, and `npm run build` pass.
- **Failing tests kept:** the proxy test was written to fail before the fix, and no test was deleted, skipped, or weakened.
- **Manual, local (Chrome, desktop):** create, format, refresh, import, share, Nora read-only, and switching demo accounts.
- **Manual, live Supabase and Vercel (Chrome, desktop):** sign-in, create, format, refresh, import, and share. Signed-out redirects, a generic 401 for bad credentials, 403 for a cross-origin request, and the demo cookie ignored in production.
- **Data safety:** the local database was backed up and the copy verified before a stuck dev server was stopped.

Not verified: Firefox, Safari, Edge, tablet, and phone; performance (no LCP, INP, or CLS numbers); and signing in as the editor or viewer on the live site.

My view of AI in engineering

AI can help move through repetitive setup, documentation, test ideas, and debugging faster. I still need to understand the code I ship, decide which features matter, verify edge cases, and own the final implementation. For this assignment, I want the reviewer to see my prioritization and reasoning as clearly as the product itself.